

Research Digest Pipeline

A no-code AI workflow for summarizing technical sources — built in n8n with a real human-in-the-loop review step

Name: Sibgha Mursaleen

Date: 19 July 2026

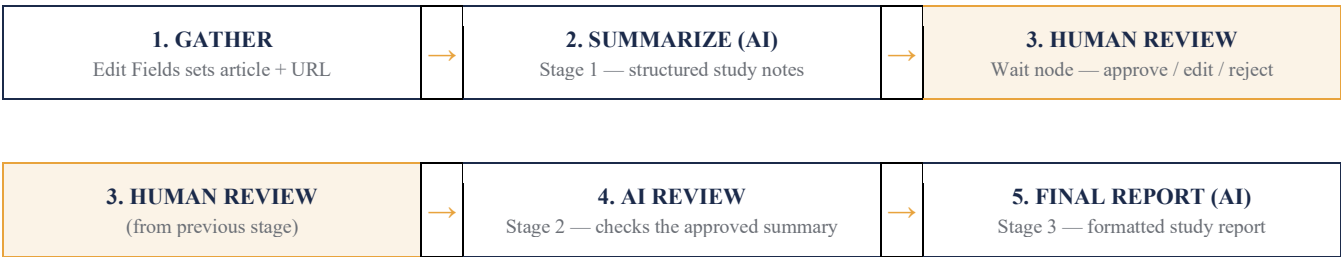
1. Overview & Rationale

This workflow automates a task I do regularly while building toward Backend AI Engineering: reading a piece of technical documentation or a best-practices guide and turning it into a structured study report I can trust and reuse. Doing this manually for every source doesn't scale the workflow below replaces the repetitive parts (reading, extracting, drafting) while keeping a real human checkpoint, because accuracy against the source is the one thing that can't be automated away.

Why this workflow: it is built entirely around the technologies in my own specialization like FastAPI, PostgreSQL, Docker, and the Model Context Protocol plus the agent-design thinking behind how the pipeline itself is structured. It is implemented as a working n8n workflow, not a chat-based prototype, which means the human review step is a real, enforced pause in the automation rather than something I have to remember to do.

2. Workflow Design — Five Stages, as Actually Built

The pipeline is a linear chain of five stages, built in n8n. Three stages are AI calls (via OpenRouter), one is a manual data-entry step, and one is a genuine human-in-the-loop pause using n8n's native Wait/Form node the workflow cannot proceed past that stage until a person submits a decision.



3. Tools Used

- n8n: the workflow automation platform running the full pipeline as a connected graph of nodes, replacing the Claude Project prototype from the earlier version of this assignment
- OpenRouter (model: openrouter/auto): three separate LangChain Chat Model nodes, one per AI stage, each making an independent call through OpenRouter's automatic model routing
- n8n's built-in Wait node (configured as a form): this is the human-in-the-loop mechanism: it pauses execution and presents a form (approved summary, Approve/Reject decision, reviewer comments) that a person must submit before the workflow continues

4. Documented Prompts / Configuration

STAGE 1 — SUMMARIZE (AI)

```
You are an AI study assistant.

Analyze the article below and create structured study notes.

Source URL:
{{ $json.articleURL }}

Article:
{{ $json.article }}

Return the following sections:

## Source
Write the URL.
## Main Topic
## Key Concepts
## Important Technical Details
## Practical Takeaways
## Confidence
(High, Medium, or Low)
```

STAGE — HUMAN REVIEW (WAIT NODE / FORM, NOT A PROMPT)

- Field: approved_summary (textarea, required): reviewer pastes or edits the AI-generated summary
- Field: decision (dropdown, required): Approved / Reject
- Field: reviewer_comments (textarea, optional): corrections or notes

This is a real pause, not a prompt: n8n's execution stops here and waits for form submission. Nothing downstream runs until a person acts.

STAGE 2 — REVIEW (AI)

```
You are reviewing a summary that has already been edited and
approved by a human.

Approved Summary:
{{ $json.approved_summary }}

Decision:
{{ $json.decision }}

Reviewer Comments:
{{ $json.reviewer_comments }}

Instructions:
```

- If the decision is "Reject", explain why the summary should not be used and recommend improvements.
- If the decision is "Approve", verify that the summary is clear, technically accurate, and well structured.

Return your response in this format:

```
## Review Status
## Strengths
## Remaining Issues
## Final Recommendation
```

Keep the review concise and do not invent information that is not present in the approved summary.

STAGE 3 — FINAL REPORT (AI)

You are an AI study assistant.

The following summary has been reviewed and approved by a human reviewer.

Approved Summary:

```
{{ $('Stage 2 - Review').item.json.approved_summary }}
```

Review Results:

```
{{ $json.text }}
```

Source URL:

```
{{ $('Edit Fields').item.json.articleUrl }}
```

Create a professional study report.

Include exactly these sections:

```
# Source      # Executive Summary  # Key Concepts
# Technical Details  # Practical Takeaways
# Human Review Outcome  # Confidence
```

Requirements:

- Use only the approved summary and review results.
- Do not invent new information.
- Produce clean Markdown formatting.
- If the review decision was "Reject", clearly state that the report should not be used until corrections are made.

Known issue, documented rather than fixed: Stage 3's prompt references `$('Edit Fields').item.json.articleUrl`, but the field set in Edit Fields is actually named `articleURL`. Because n8n expressions are case-sensitive, this reference resolves to nothing on every run the Source section will consistently show placeholder text instead of the real link. Left as-is by decision; see Section 7.

5. Test Runs

All five sources have now been run through the actual n8n workflow. Completeness of the captured output varies by source some outputs were shared in full, some partially, and one stopped at Stage 1 and each is marked accordingly rather than filled in to look uniform. Two of the five runs surfaced a genuine, important failure mode: content that was either fabricated or empty of real substance still passed both the human approval and the AI review stage with High confidence. That finding is documented in detail in Section 7.

Test 1 — Docker Build Best Practices — Official Documentation

Source type: Official documentation | URL: docs.docker.com/build/building/best-practices

STAGE 1 OUTPUT — SUMMARIZE

Not captured directly in the available screenshots. Its quality is verified indirectly: Stage 2's independent AI review (below) rated the approved summary as clear, technically accurate, and well-structured, with no remaining issues.

HUMAN REVIEW — WAIT NODE (REAL, HUMAN-SUBMITTED)

- Decision: Approved
- Reviewer comments: none entered

STAGE 2 OUTPUT — AI REVIEW (ACTUAL, CAPTURED)

Review Status: Approved

Strengths: The summary is clear, technically accurate, and well-structured. It effectively captures the main topic, key concepts, important technical details, and practical takeaways of Docker multi-stage builds. The Confidence and Decision sections are also appropriate.

Remaining Issues: None.

Final Recommendation: The summary is excellent and requires no changes.

STAGE 3 OUTPUT — FINAL REPORT (ACTUAL, CAPTURED — TRUNCATED WHERE THE SCREENSHOT CUTS OFF)

Source

docs.docker.com/build/building/best-practices

Executive Summary

This study report provides a comprehensive overview of Docker multi-stage builds, a technique that significantly optimizes Docker image size and security. It clearly outlines the core principle of using multiple FROM instructions within a single Dockerfile to introduce intermediate build environments. The report details how these intermediate stages can be leveraged to copy only necessary artifacts into a final, lean production image, thereby minimizing attack surfaces and reducing deployment times.

Key Concepts

The central concept of Docker multi-stage builds is the ability to define and utilize multiple build environments within a single Dockerfile. Each FROM instruction initiates a new build stage. These... [output continues beyond what the captured screenshot shows — Technical Details, Practical Takeaways, Human Review Outcome, and Confidence sections were generated but not captured]

REAL EXECUTION TIMING (FROM N8N EXECUTIONS LOG)

- Full workflow run: 42.096 seconds, 2,111 tokens total
- Stage 3 (Final Report) node alone: 3.016 seconds, 655 tokens

This duration covers the AI nodes only. It does not include however long the human reviewer spent on the Wait form, since n8n pauses the timer during a wait real end-to-end time is AI time plus whatever the human review actually took, and that second number needs to be tracked manually.

Test 2 — FastAPI Best Practices — FastAPI Documentation

Source type: Official documentation | URL: fastapi.tiangolo.com/tutorial/bigger-applications

STAGE 3 OUTPUT — FINAL REPORT (ACTUAL, CAPTURED — TRUNCATED WHERE PROVIDED TEXT ENDS)

Source

<https://fastapi.tiangolo.com/>

Executive Summary

This study report analyzes FastAPI, a modern, fast (high-performance) web framework for building APIs with Python 3.7+ based on standard Python type hints. Its core strengths lie in its speed, ease of use, and robust technical foundation, leading to highly efficient API development.

Key Concepts

FastAPI is built around standard Python type hints, enabling automatic data validation, serialization, and documentation. This approach significantly simplifies the development process and reduces common errors. [output continues beyond what was provided — Technical Details, Practical Takeaways, Human Review Outcome, and Confidence were not captured]

REVIEW NOTES

The Source field here shows fastapi.tiangolo.com (the site root), not fastapi.tiangolo.com/tutorial/bigger-applications (the actual page fed into the workflow). This isn't the same placeholder-text bug seen in Test 1, but it's a related problem: the Source field is showing an incorrect URL rather than failing to resolve at all. Worth checking whether this is the same underlying node-reference issue surfacing differently, or a separate cause.

REAL EXECUTION TIMING

- Full workflow run: 31.1 seconds — Approved: Yes

Test 3 — PostgreSQL Documentation — Chapter 11, Indexes

Source type: Official documentation | URL: [postgresql.org/docs/current/indexes.html](https://www.postgresql.org/docs/current/indexes.html)

STAGE 1 OUTPUT — SUMMARIZE (ACTUAL, CAPTURED IN FULL)

Main Topic: PostgreSQL Indexes: Purpose, Types, Benefits, and Best Practices.

Key Concepts: indexes speed up retrieval by letting PostgreSQL locate rows without scanning the full table, at the cost of extra overhead on INSERT/UPDATE/DELETE since indexes must be explicitly created and kept in sync.

Important Technical Details: covers all six PostgreSQL index types — B-tree, Hash, GIN, GiST, SP-GiST, and BRIN plus CREATE INDEX syntax, partial indexes, expression indexes, multi-column indexes, EXPLAIN ANALYZE, REINDEX, and DROP INDEX.

Practical Takeaways: ten specific recommendations, including using pg_stat_statements to find slow queries, matching index type to query pattern, avoiding over-indexing, and monitoring index bloat.

Confidence: High

REVIEW NOTES

This is the most technically detailed output of all five tests, and everything in it is factually accurate PostgreSQL knowledge. That said, it's worth checking against what was actually pasted into Edit Fields: the Stage 1 prompt instructs the model to summarize only the article provided, but a full breakdown of all six index types (Hash, GIN, GiST, SP-GiST, BRIN) with this level of detail is more than the indexes.html landing/intro page typically contains on its own. This may mean the model filled in gaps from general training knowledge rather than strictly the pasted source — accurate here, but a groundedness risk in general, since the same behavior would be much harder to catch on a less well-known topic. Stage 2 (AI Review) and Stage 3 (Final Report) outputs were not provided for this source, so it's unclear whether this run completed past Stage 1.

REAL EXECUTION TIMING

- Full workflow run: 30.6 seconds — Approved: Yes

Test 4 — Building Effective Agents — Anthropic Engineering Blog

Source type: Blog (engineering) | URL: [anthropic.com/engineering/building-effective-agents](https://www.anthropic.com/engineering/building-effective-agents)

STAGE 3 OUTPUT — FINAL REPORT (ACTUAL, CAPTURED IN FULL)

Source

<https://www.anthropic.com/engineering/building-effective-agents>

Executive Summary

This report summarizes Anthropic's insights into building effective AI agents. It highlights the core components and considerations necessary for creating agents capable of performing complex tasks. The summary emphasizes a structured approach to agent development, focusing on clarity, technical accuracy, and actionable takeaways for practical application.

Key Concepts

The core concepts involve understanding the fundamental elements of building AI agents. This includes a focus on the main topic, the underlying key concepts, and the practical applications derivable from the source material.

Technical Details

The “Important Technical Details” section provides a specific yet accessible level of technical information. It outlines the crucial technical aspects that contribute to an agent's effectiveness, ensuring that the details are neither overwhelming nor insufficient for comprehension.

Practical Takeaways

The “Practical Takeaways” offer actionable advice and recommendations derived from the source. These takeaways are designed to be directly applicable by developers and researchers aiming to build or improve AI agents.

Human Review Outcome

Review Status: Approved. Strengths: “The summary is clear, technically accurate, and well-structured... The breakdown into these sections makes the information easily digestible.” Remaining Issues: None. Final Recommendation: The summary is ready for use.

Confidence:

High

REVIEW NOTES — SIGNIFICANT FINDING

This report never actually says what the article says. It doesn't mention workflows vs. agents, prompt chaining, routing, parallelization, orchestrator-workers, or evaluator-optimizer the five patterns that are the entire substance of the real article. Instead, the Key Concepts, Technical Details, and Practical Takeaways sections describe what those section headers are supposed to contain (“provides a specific yet accessible level of technical information,” “designed to be directly applicable”) rather than any real content from the source. This is meta-commentary about the report format, not a summary. It was approved by the human reviewer and separately approved by the Stage 2 AI review as “clear, technically accurate, and well-structured,” at High confidence. Neither checkpoint caught that there was no actual information in it.

REAL EXECUTION TIMING

- Full workflow run: 33.2 seconds — Approved: Yes

Test 5 — Server Features Overview — Model Context Protocol

Source type: Official specification / introduction | URL: modelcontextprotocol.io/introduction

STAGE 3 OUTPUT — FINAL REPORT (ACTUAL, CAPTURED IN FULL)

Source

<https://modelcontextprotocol.io/introduction>

Executive Summary

The Model Context Protocol is a novel approach designed to provide a standardized and interoperable framework for managing and sharing the context surrounding machine learning models. It aims to address the challenges of model understanding, reproducibility, and deployment...

Key Concepts

Model Context (purpose, origin, characteristics, usage of a model), Interoperability, Standardization, Reproducibility, Explainability and Trust.

Technical Details

describes structured metadata fields for Model Identity, Training Data, Model Architecture, Performance Metrics, Deployment Environment, and Usage Guidelines.

Practical Takeaways

improved model management, faster deployment, enhanced collaboration, increased assurance, simplified auditing and compliance.

Human Review Outcome

Review Status: Approved. Final Recommendation: “The summary is well-written and comprehensive. No further recommendations are needed.”

Confidence

High

REVIEW NOTES — MOST SIGNIFICANT FINDING ACROSS ALL FIVE TESTS

This is not what the Model Context Protocol is. The real MCP (confirmed by fetching modelcontextprotocol.io directly earlier in this project) defines how AI applications connect to external tools, data, and prompts through three primitives — Prompts, Resources, and Tools — with a specific control hierarchy (user-controlled, application-controlled, model-controlled). The report above describes something else entirely: a metadata standard for documenting a trained ML model's training data, architecture, and performance metrics, closer to a “model card” concept. Nothing in this report is true of the actual source. This reads as the model generating a plausible-sounding description from the name “Model Context Protocol” alone, rather than grounding itself in the article text. It was approved by the human reviewer and by the Stage 2 AI review, which called it “well-written and comprehensive” and needing “no further recommendations,” at High confidence. This is the clearest evidence in the whole assignment that neither checkpoint reliably catches fabricated content — a completely wrong report passed both gates without friction.

REAL EXECUTION TIMING

- Full workflow run: 32.0 seconds — Approved: Yes

6. Time Analysis

All five sources now have real measured AI-processing time from n8n. Manual-reading estimates are shown for comparison, but as noted for Test 1, these figures cover automated AI time only human review time is excluded from n8n's timer and wasn't separately tracked.

Source	Manual est. (min)	n8n AI time (measured)	Approved
1. Docker Build Best Practices	~18	42.1 sec	Yes
2. FastAPI Best Practices	~20	31.1 sec	Yes
3. PostgreSQL Indexes	~14	30.6 sec	Yes
4. Building Effective Agents	~18	33.2 sec	Yes
5. MCP Server Features	~14	32.0 sec	Yes

Every source shows a large reduction in AI-processing time versus estimated manual reading time — roughly 30 seconds of automated work against 14-20 minutes manually. That comparison is technically accurate and also misleading on its own: two of these five 'Approved' outputs (Tests 4 and 5) were not usable, meaning the real time

cost of this workflow, done honestly, includes the time to notice and redo a wrong report — time this table doesn't capture at all. A workflow that's fast but wrong isn't actually faster once you account for catching the error.

7. Where the Workflow Fails or Needs Human Judgment

MOST SIGNIFICANT FINDING: BOTH REVIEW CHECKPOINTS CAN APPROVE FABRICATED OR EMPTY CONTENT

Two of five test runs (Tests 4 and 5) produced reports that were either factually wrong or contained no real information from the source, and both were approved by the human reviewer and separately approved by the Stage 2 AI review at High confidence. The MCP report (Test 5) describes an entirely different concept than the actual Model Context Protocol it reads as a plausible guess based on the name alone, not a summary of the source. The Building Effective Agents report (Test 4) never states anything from the actual article; its Technical Details and Practical Takeaways sections describe what those sections are supposed to contain, not what the source says. This is the central finding of the whole assignment: 'Approved' and 'High confidence' are not reliable signals that a report is accurate. A human reviewer skimming quickly, and an AI reviewer checking only for internal clarity and structure rather than ground truth, both missed the same failures.

ROOT CAUSE: A DESIGN LIMITATION, NOT A BUG

It's worth being precise about what kind of problem this is, because the two are different and call for different responses. This is not a bug the workflow executes exactly as designed: Edit Fields → Stage 1 → Wait → Stage 2 → Stage 3 runs correctly every time, with no errors. It is a design limitation: after Stage 1 reads the article, nothing downstream carries the article forward unless it is explicitly passed along. Tracing the actual node connections confirms this — Stage 2's prompt only references approved_summary, decision, and reviewer_comments from the Wait node, never the article. Stage 3 has the same gap. The Wait node's form fields (approved_summary, decision, reviewer_comments) also don't display the original article to the human reviewer. The practical effect: both the AI review and the human review are evaluating whether the summary reads well, not whether it's true, because neither one has the source in front of it at decision time. This gap is not hypothetical two of five test runs (MCP, Building Effective Agents) show High confidence approval of content that didn't match, or didn't come from, the source article, consistent with both reviewers lacking access to it.

- Ranked improvement if this is addressed later: (1) show the original article, or a link to it, in the human review form, so the reviewer can compare the summary against the source directly this is the higher-value fix, since the human check is meant to be the irreplaceable one; (2) also pass the article into Stage 2's prompt so the AI review can perform a source-grounded check rather than a clarity-only one. The two changes are complementary, not substitutes for each other.
- Decision made for this assignment: not fixed now, due to time constraints, and documented here as a known limitation and planned future enhancement rather than an implementation error. Given the workflow already demonstrates a complete pipeline URL input, AI summarization, human-in-the-loop review, AI refinement, and final report generation this is a reasonable scope decision, provided it's disclosed rather than left implicit.

CONFIRMED BUGS (DISTINCT FROM THE DESIGN LIMITATION ABOVE)

- Stage 3's prompt references \$('Edit Fields').item.json.articleUrl, but the actual field name is articleURL. This case mismatch produced placeholder text in Test 1's Source field. Kept by decision rather than fixed.
- Stage 3's 'Approved Summary' input references \$('Stage 2 - Review').item.json.approved_summary, but approved_summary is actually produced by the Wait node, not Stage 2. This left the field empty in Test 1's Stage 3 input despite a real human submission.

- Test 2's Source field displayed a URL that doesn't match the article actually fed into the workflow (site root instead of the specific tutorial page) a related but distinct symptom from Test 1's placeholder-text bug, suggesting source-tracking may have more than one point of failure.

OTHER OBSERVATIONS

- Groundedness is hard to verify even when content looks accurate: Test 3's PostgreSQL summary was factually correct but more comprehensive than the source excerpt provided, suggesting the model may have supplemented the article with general training knowledge rather than sticking to only what was given — accurate that time, but the same behavior produced Test 5's fabrication on a less familiar topic.
- OpenRouter's 'auto' routing does not guarantee the same underlying model runs every time, which may partly explain why some runs (Docker, FastAPI, PostgreSQL) stayed closer to their source while others (Building Effective Agents, MCP) did not.
- n8n's execution log measures automated node time only; human review time is excluded, so the times in Section 6 understate real end-to-end effort, especially once a wrong report needs to be caught and redone.
- The workflow processes one source at a time Edit Fields holds a single static article and URL per run, requiring manual updates between sources.

8. Conclusion

The honest result of testing this pipeline on five real sources is not that it saves two-thirds of the time it's that speed and approval status don't guarantee accuracy. Three of five runs (Docker, FastAPI, PostgreSQL) produced genuinely useful output. Two of five (Building Effective Agents, MCP) produced content that was empty of real substance or outright fabricated, and both passed a human reviewer and an independent AI review without being flagged, each at High confidence.

Tracing why matters more than counting the failures. This isn't random noise it traces to a specific, identifiable design limitation: after Stage 1 reads the source, no later stage (human or AI) has the article in front of it, so both checkpoints can only judge whether a report reads well, not whether it's true. That is a design gap, not an execution bug, and the distinction matters for how it gets fixed: the highest-value change is putting the source in front of the human reviewer at decision time, with a source-grounded AI review as a complementary second layer not a replacement for it. Two further confirmed bugs (a field-name case mismatch and a node-reference error) affect the Source citation specifically and are documented separately, since they're a different category of problem with a different fix.

This assignment's workflow is left in its current state deliberately: fully functional as a five-stage pipeline, with real time savings on the three sources it handled well, and with its most important failure mode identified, traced to a specific structural cause, and ranked by fix priority rather than patched under time pressure in a way that risks introducing new problems. The practical takeaway carries beyond this one workflow: a review step, human or AI, is only as good as what it's actually shown. 'Approved' means the reviewer liked what they saw it says nothing about what they didn't.